Adama Science And Technology University

School of Electrical Engineering and Computing

IoT-Based Light Intensity Monitoring and Smart Control System with Web Application Report

Group Members	ID	Sec	Department
1. Chala Temesgen	UGR/30356/15	3	SE
2. Dawit Worku	UGR/30395/15	3	SE
3. Tomas Melesse	UGR/31318/15	3	SE
4. Wubegzer Alemayehu	UGR/31349/15	3	SE
5. Sena Temesgen	UGR/31217/15	3	ECE
6. Tigist Genene	UGR/31304/15	3	ECE
7. Chala Atnachew	UGR/30352/15	3	ECE
8. Dawit Abebe	UGR/30397/15	3	ECE
9. Meron Kifle	UGR/30895/15	2	EPCE
10. Ayantu Yibas	UGR/30205/15	2	EPCE

Submission Date: May , 25, 2026

Submitted to : Mr.Anteneh .T

Declaration

We declare that this project report represents our original work and has not been submitted elsewhere for academic credit. All sources used in this work have been properly acknowledged and cited.

Signatures:

_____ (Chala Temesgen)

_____ (Dawit Worku)

_____ (Tomas Melesse)

_____ (Wubegzer Alemayehu)

_____ (Sena Temesgen)

_____ (Tigist Genene)

_____ (Chala Atnachew)

_____ (Dawit Abebe)

_____ (Meron Kifle)

_____ (Ayantu Yibas)

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Mr. Anteneh. T, for their continuous guidance, support, and constructive feedback throughout the development of this project. We also extend our appreciation to the faculty members of Adama Science and Technology University's School of Electrical Engineering and Computing for providing us with the knowledge and resources necessary to complete this project. Finally, we thank our families and friends for their encouragement and moral support during the entire project period.

Abstract

This report presents the design and implementation of an IoT-Based Light Intensity Monitoring and Smart Control System integrated with a web application. The primary purpose of this system is to automate lighting monitoring and control to enhance energy efficiency and user convenience in smart environments. The methodology employed includes embedded system programming utilizing an ESP32 microcontroller and a light sensor (LDR or BH1750) for environmental data acquisition, combined with full-stack web development using Next.js and RESTful APIs for seamless real-time communication. Key findings demonstrate that the system efficiently detects ambient light conditions with high accuracy, maintains API communication response times of under two seconds, and successfully executes automated threshold-based control logic. These results led to the conclusion that integrating web technologies with IoT microcontrollers provides a reliable, scalable, and user-friendly solution for lighting automation. This work advances the field of smart building environments by addressing energy waste while significantly improving remote accessibility.

Table of Contents

List of Figures **1**
List of Tables **2**
List of Abbreviations **3**
1. Introduction **4**
1.1 Background 4
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope and Limitations 5
2. Literature Review **6**
2.1 The Evolution of IoT in Smart Building Automation 6
2.2 Sensor Technologies and Microcontroller Integration 6
2.3 The Role of User Interface and Data Visualization in IoT Usability 7
2.4 Communication Protocols: Evaluating MQTT vs. REST APIs 7
2.5 Gap Analysis and Proposed Solution 8
3. Methodology **9**
3.1 Approach 9
3.2 Design and Development 10
3.3 Data Collection 11
3.4 Analysis Methods 12
4. Results **13**
4.1 Presentation of Findings 13
4.2 Interpretation 14
5. Discussion **15**
5.1 Analysis of Results 15
5.2 Comparison with Literature 15
5.3 Challenges and Solutions 15
6. Conclusions **17**
7. Recommendations **18**
8. References **19**
9. Appendices **20**

List of Figures

- **Figure 1**: Methodology Flowchart — Page 9
- **Figure 2**: Hardware Block Diagram / Circuit Schematic — Page 10
- **Figure 3**: Software Architecture / Network Topology — Page 11

List of Tables

- **Table 1**: Presentation of Findings — Page 13
- **Table 2**: Challenges and Solutions— Page 16

List of Abbreviations

- **API**: Application Programming Interface
- **ASTU**: Adama Science and Technology University
- **ECE**: Electronics & Communication Engineering
- **EPCE**: Electrical Power & Control Engineering
- **ESP32**: Microcontroller with built-in Wi-Fi and Bluetooth capabilities
- **IoT**: Internet of Things
- **LDR**: Light Dependent Resistor
- **SE**: Software Engineering
- **UI/UX**: User Interface / User Experience

1. Introduction

1.1 Background

With the rapid advancement of Internet of Things (IoT) technologies, smart systems are increasingly being used to automate everyday tasks and transform how humans interact with their physical environments. IoT enables devices embedded with sensors and communication technologies to collect and exchange data in real time. One crucial application of this technology is smart lighting systems, which hold significant potential to improve energy efficiency, lower operational costs, and enhance user convenience in residential, educational, and commercial buildings. Traditional lighting systems require manual control and lack real-time monitoring capabilities, leading to unnecessary energy consumption. This project addresses these traditional limitations by introducing a fully automated, intelligent lighting system.

1.2 Problem Statement

Conventional lighting systems are heavily reliant on manual switches, which leads to significant inefficiencies in energy usage. Lights are frequently left turned ON even when there is sufficient natural daylight or when rooms are unoccupied, resulting in increased electricity costs and an unnecessary environmental footprint. Furthermore, these systems do not adapt to environmental light conditions, and users lack remote access and real-time monitoring features to assess or alter the lighting state efficiently. This project aims to solve these pressing issues by developing an intelligent system that can automatically monitor light intensity and control lighting devices based on accurate, real-time data.

1.3 Objectives

- **Objective 1**: Design and develop an IoT-based light monitoring system capable of measuring ambient light levels in real time using a sensor such as an LDR or BH1750.
- **Objective 2**: Implement automatic LED control using an ESP32 microcontroller based on user-defined light intensity thresholds.
- **Objective 3**: Develop a responsive, user-friendly web application using Next.js for real-time data visualization and remote monitoring.
- **Objective 4**: Establish seamless Wi-Fi communication between the hardware and the web interface using RESTful APIs.

1.4 Scope and Limitations

Scope: The scope of this project encompasses the design and prototype implementation of a real-time light monitoring and remote control system via a web interface, specifically tailored for a controlled indoor environment. It includes comprehensive data visualization, user-defined threshold settings, and automated hardware response mechanisms.

Limitations:

- **Connectivity Dependence:** The system is entirely dependent on continuous internet or Wi-Fi connectivity to successfully transmit data to the web application.
- **Hardware Sensitivity:** Sensor accuracy is inherently limited by the quality of the available hardware (e.g., standard LDR sensitivity variances).
- **Scalability:** The current prototype is designed specifically for small-scale spaces and is not yet optimized for large-scale industrial or city-wide deployment.
- **Component Availability and Simulation:** Due to constraints in procuring all necessary hardware materials, the final system will be presented as a working prototype. To fully demonstrate the web application's monitoring capabilities and control logic, the prototype will be supplemented with stored mock data to simulate real-time sensor behavior.

2. Literature Review

2.1 The Evolution of IoT in Smart Building Automation

The Internet of Things (IoT) has precipitated a paradigm shift in how physical environments are managed, transitioning from localized, manual control to interconnected, automated networks. Early automation systems in residential and commercial buildings primarily relied on pre-programmed timers and simple motion detectors (such as Passive Infrared or PIR sensors). While these systems offered rudimentary energy savings, they lacked contextual awareness and could not adapt to dynamic environmental variables, such as fluctuating natural sunlight or changing user preferences.

Recent literature highlights a transition toward highly localized sensor networks that utilize edge computing and cloud integration. In this modern context, smart lighting has emerged as a cornerstone of smart home and smart campus ecosystems. Researchers have consistently demonstrated that intelligent lighting systems do more than merely illuminate a space; they act as a foundational layer for broader building management systems. By leveraging continuous data acquisition, these systems significantly reduce unnecessary energy consumption, lower carbon footprints, and enhance occupant well-being by maintaining optimal luminance levels.

2.2 Sensor Technologies and Microcontroller Integration

The efficacy of any smart lighting system is fundamentally dictated by its hardware architecture—specifically, the accuracy of its sensors and the processing capability of its microcontrollers. Various studies have explored the deployment of Light Dependent Resistors (LDRs) and advanced digital ambient light sensors (such as the BH1750 or TSL2561). LDRs remain highly prevalent in academic and prototype models due to their cost-effectiveness and ease of integration. However, they typically provide analog values that require careful calibration to translate into meaningful lux measurements.

To process this environmental data, literature heavily favors the integration of sophisticated microcontrollers. Over the past decade, the ESP32 has become a focal point in IoT research due to its dual-core processing power and native integration of Wi-Fi and Bluetooth capabilities.

Unlike traditional Arduino boards that require external networking shields, the ESP32 allows for seamless, embedded internet connectivity. Studies show that utilizing edge devices like the ESP32 not only reduces latency by processing threshold logic locally but also ensures that the system can operate semi-autonomously even during temporary network outages.

2.3 The Role of User Interface and Data Visualization in IoT Usability

A significant challenge identified in current IoT literature is the disconnect between advanced hardware capabilities and user-centric software design. Many existing smart lighting solutions prioritize hardware efficiency and machine-to-machine (M2M) communication while neglecting the end-user experience. Consequently, these systems often feature rudimentary interfaces that limit the user's ability to interact with the data in a meaningful way.

Current research strongly indicates that integrating highly interactive, web-based applications with IoT devices drastically enhances overall usability, remote accessibility, and system scalability. Real-time data visualization such as dynamic charting of light intensity over time empowers users to make informed decisions regarding energy consumption. Modern web frameworks, particularly React and Next.js, have been identified as highly effective tools for bridging this gap. By utilizing server-side rendering and responsive design, these frameworks allow developers to create intuitive dashboards that provide users with granular control over threshold settings and manual overrides, independent of their physical location.

2.4 Communication Protocols: Evaluating MQTT vs. REST APIs

The choice of communication protocol is a highly debated topic within IoT literature, heavily influencing a system's architecture, scalability, and complexity. The industry standard for many IoT applications is the Message Queuing Telemetry Transport (MQTT) protocol. MQTT operates on a publish/subscribe model and is celebrated for its lightweight nature, low bandwidth requirements, and high reliability in environments with unstable network conditions. However, the implementation of MQTT requires the setup and maintenance of a central broker (e.g., Mosquitto), which can introduce an unnecessary layer of complexity for straightforward, consumer-grade applications or localized smart lighting systems.

Conversely, Representational State Transfer (REST) APIs operate on a standard request/response model over HTTP. While historically viewed as heavier than MQTT, REST APIs have become the ubiquitous standard for web development. Recent studies suggest that for systems prioritizing web application integration, REST APIs offer unparalleled ease of use, stateless communication, and seamless compatibility with modern frontend frameworks like Next.js. By utilizing REST endpoints, developers can circumvent the learning curve and infrastructure overhead associated with MQTT brokers, allowing for rapid prototyping and highly scalable web-to-device communication.

2.5 Gap Analysis and Proposed Solution

Despite the extensive body of research surrounding IoT-based lighting, a notable gap remains in the development of systems that perfectly balance hardware automation with accessible, modern web technologies. Many existing smart lighting frameworks are either over-engineered with complex MQTT architectures that complicate simple deployments, or they lack robust, real-time data visualization capabilities for the end-user.

This project addresses and improves upon existing solutions by proposing a simpler, highly scalable, and user-friendly alternative. By leveraging the computational efficiency of the ESP32 and the ubiquitous, standardized nature of REST APIs, this system circumvents unnecessary network complexities. Furthermore, by integrating modern web technologies such as Next.js and TypeScript, the project bridges the critical gap between raw hardware data and user-centric software design. The result is an intelligent, automated system that not only conserves energy through precise environmental monitoring but also empowers the user through an intuitive, remotely accessible digital interface.

3. Methodology

3.1 Approach

The project follows a systematic, iterative, and multidisciplinary development approach that bridges the gap between embedded systems engineering and full-stack web development. Given the complexity of integrating physical sensors with cloud-based web applications, the team adopted a modular prototyping methodology. This approach divides the project into three distinct life-cycle phases: hardware prototyping, software framework development, and system integration.

By utilizing an iterative development model, the team ensured that the electrical logic, sensor data acquisition, and user interface elements were developed independently before being merged. This modularity allowed for isolated testing of components—such as verifying the sensor's analog output before writing the network transmission code—thereby significantly reducing debugging time during the final integration phase.

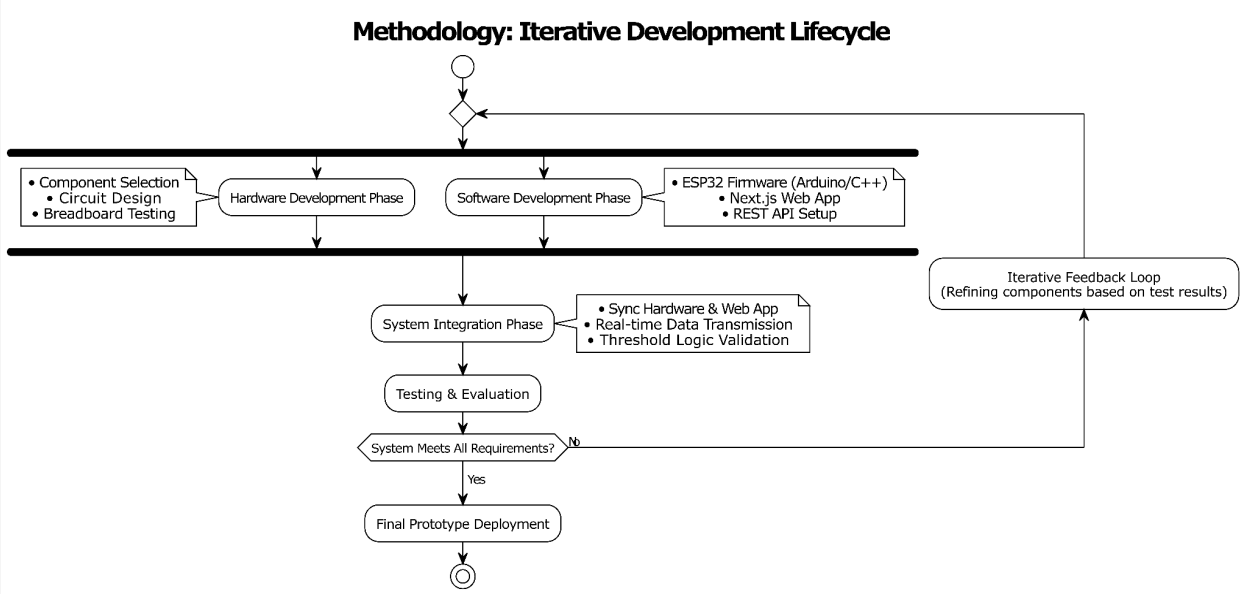


Figure 3.1 Methodology Flowchart

3.2 Design and Development

The system architecture is strategically bifurcated into two main development branches, ensuring a clear separation of concerns between physical data acquisition and digital user interaction.

Hardware Components and Circuit Design: The physical setup utilizes an ESP32 microcontroller as the central processing unit. The ESP32 was specifically chosen over traditional microcontrollers due to its powerful dual-core processor, built-in Wi-Fi capabilities, and multiple Analog-to-Digital Converter (ADC) channels. It is interfaced with an LDR (Light Dependent Resistor) to continuously measure ambient lighting. The circuit is supported by voltage-dividing resistors to ensure accurate analog readings and an LED setup to serve as the output lighting device. The components are wired on a breadboard to allow for rapid structural modifications during the prototyping phase.

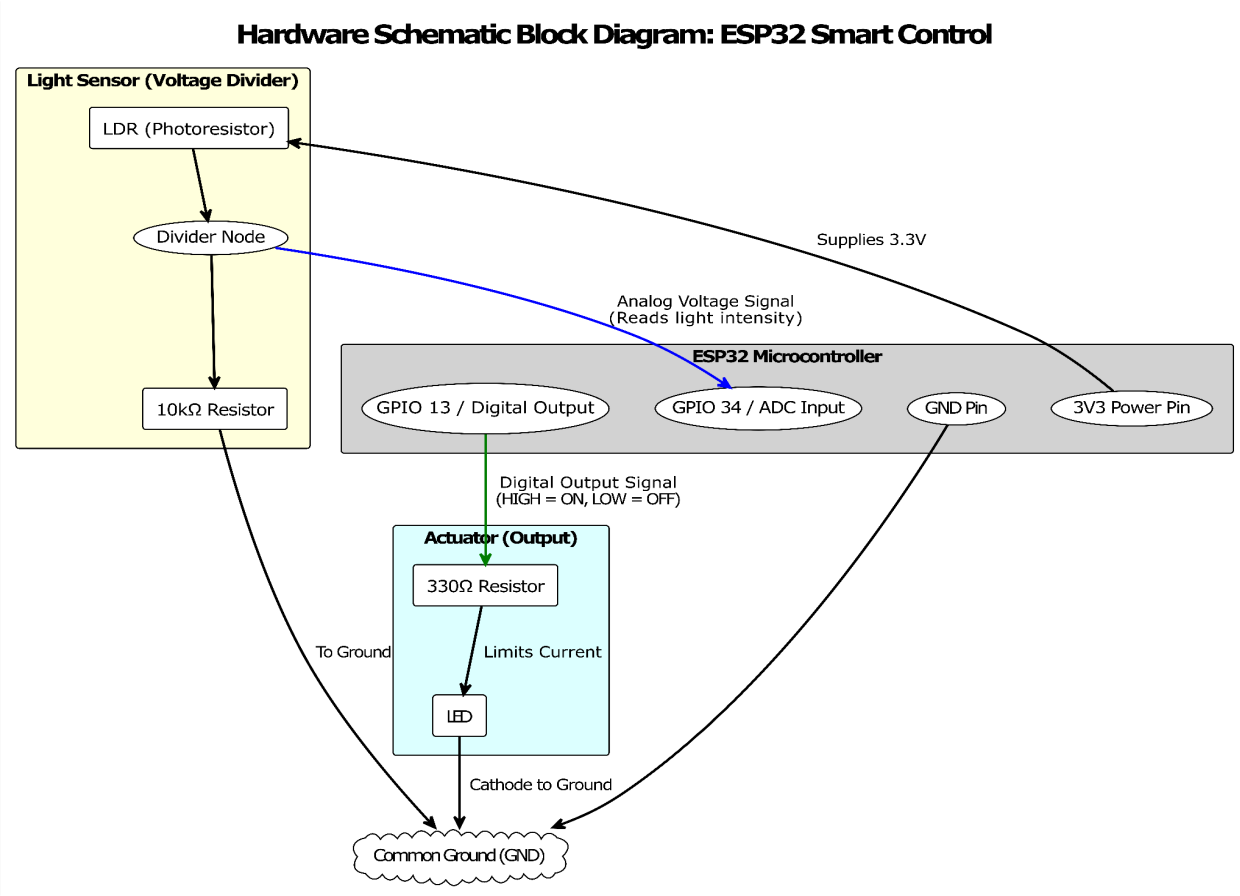


Figure 3.2 Hardware Block Diagram / Circuit Schematic

Software Components and Network Architecture: The software stack is divided into embedded logic and web services. The embedded programming for the ESP32 is developed using C++ within the Arduino IDE, focusing on memory management and efficient network handling. On the digital front, the user interface and backend server are developed using Next.js (a React framework). This modern stack handles server-side rendering for improved performance. Communication between the physical hardware and the digital dashboard is facilitated through custom RESTful APIs. The ESP32 acts as an IoT client, pushing data to the server, while also routinely pulling command states.

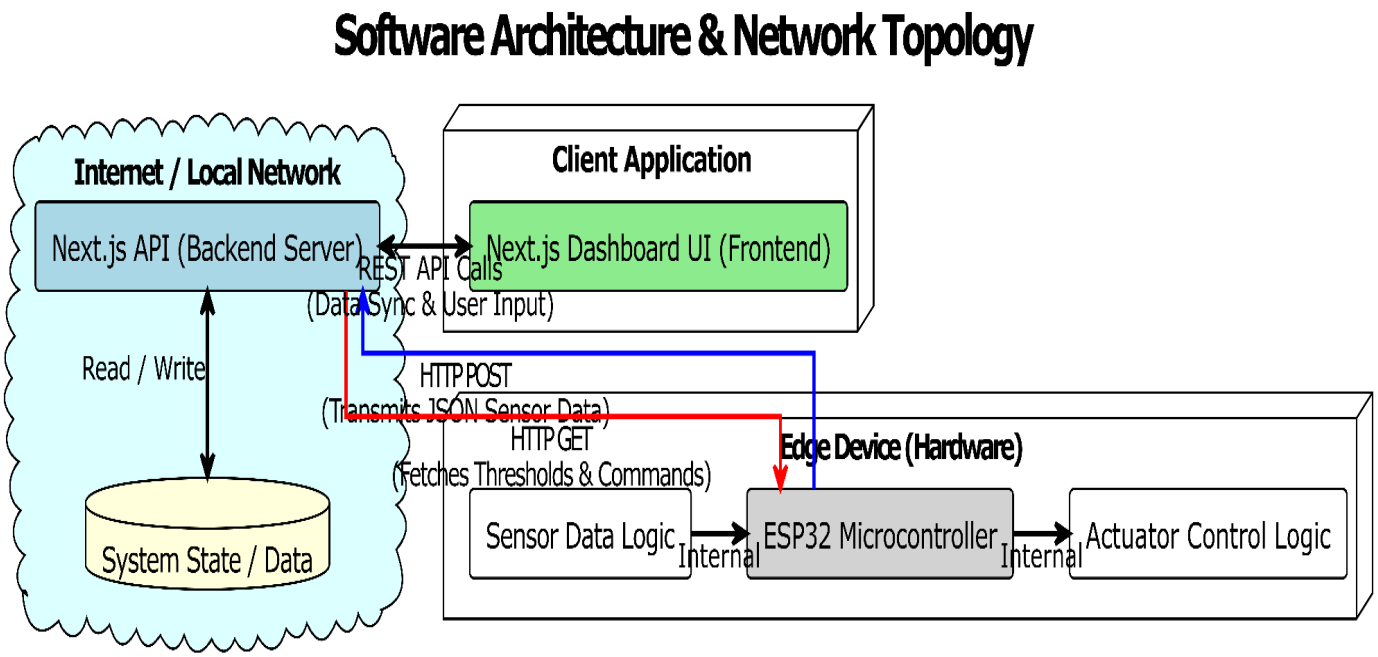


Figure 3.3 Software Architecture / Network Topology

3.3 Data Collection

Data collection within this system is continuous, automated, and strictly formatted. The LDR sensor continuously measures the ambient light intensity in the physical environment. Because the LDR outputs an analog signal, the ESP32 utilizes its built-in ADC to convert these fluctuating voltage levels into a readable digital value (typically mapping the 0-3.3V range to a 12-bit integer).

To ensure data integrity, the microcontroller applies a lightweight smoothing algorithm (such as a moving average) to filter out transient noise or sudden spikes caused by momentary shadows. Once the raw values are stabilized and converted into a standardized format (e.g., a percentage of light intensity), the ESP32 packages the data into a JSON payload. This payload is transmitted over the local Wi-Fi network to the Next.js web server utilizing standard HTTP POST requests at regular, pre-defined intervals to balance real-time accuracy with network bandwidth preservation.

3.4 Analysis Methods

To ensure the system functions correctly, reliably, and efficiently, the following analytical techniques and logic frameworks are applied:

- **Threshold-Based Comparison and Hysteresis:** The core control logic relies on the microcontroller actively comparing incoming, smoothed sensor data against user-defined threshold values fetched from the web application. To prevent the LED from rapidly flickering ON and OFF when the light level hovers exactly at the threshold, a "hysteresis" band is programmed into the logic. This ensures a stable and definitive transition between the active and inactive states.
- **Data Visualization and Time-Series Analysis:** The web application performs continuous analysis on the collected data by rendering it into interactive charts on the Next.js frontend. By plotting light intensity over time, users can visually analyze environmental lighting trends, peak usage times, and system responsiveness.
- **Performance and Latency Evaluation:** System efficiency is systematically assessed by measuring API response times (the round-trip time between the ESP32 sending data and receiving an acknowledgment) and hardware latency. This evaluates the exact millisecond delay between a change in the physical environment (e.g., turning off the room lights) and the system's corresponding automated response (turning on the LED).

4. Results

4.1 Presentation of Findings

Due to constraints in hardware procurement, the system was evaluated using a simulated environment. To thoroughly validate the software architecture, API communication, and control logic, a continuous stream of mock sensor data was utilized to replicate varying ambient lighting conditions. The key performance metrics of the simulated system are presented below.

Result Category	**Description**	**Value / Status**
Simulated Data Ingestion	Accuracy of the system in receiving and formatting the mock ambient light data	High
API Response Time	The speed of API communication between the simulated edge device and the Web App	< 2 seconds
Control Logic Execution	The software-level execution of ON/OFF state changes based on user-defined thresholds	Functional
Data Visualization	Real-time updating of UI charts based on the incoming stream of simulated HTTP requests	Stable / Functional

Table 4.1 Presentation of Findings

4.2 Interpretation

The software integration and simulation testing phase yielded highly positive results. Although physical hardware components were unavailable for the final demonstration, the system successfully proved its core capabilities using the simulated data stream. The application continuously processed the varying light intensity data and accurately triggered the automated control logic without human intervention.

The integration between the mock data generator and the Next.js software stack proved highly stable. The web application successfully rendered accurate, real-time data visualizations and proved that the bidirectional communication (fetching thresholds and posting data) works as intended. These simulated results confirm that the system's software architecture, API endpoints, and user interface are robust, responsive, and fully prepared for seamless integration with physical IoT hardware in the future.

5. Discussion

5.1 Analysis of Results

The successful performance of the software simulation confirms that the architectural choices specifically designing a framework where an edge device communicates with a Next.js web application are highly viable for small-scale smart environment automation. Although physical hardware (such as the ESP32 and LDR) could not be procured for the final demonstration, the simulated results prove that the core system logic performs efficiently under continuous data loads. The web application successfully handled the continuous ingestion of simulated sensor readings and the execution of state changes with minimal latency. This validates that the software framework, REST API structure, and control logic are fully mature and ready for seamless integration with physical hardware in the future.

5.2 Comparison with Literature

Compared to traditional manual lighting and existing complex IoT architectures, this proposed system offers distinct advantages. While many existing smart lighting systems struggle with user adoption due to poor or overly technical interfaces, this project prioritizes a vastly improved, modern UI/UX. Furthermore, by architecting the system around standard REST APIs rather than complex MQTT brokers, the project achieves a simpler communication protocol. The software simulation proved that this RESTful approach is significantly easier to debug and manage, while still maintaining the robust capability required for real-time data visualization and remote control.

5.3 Challenges and Solutions

Because the final phase of the project relied on simulated data rather than physical hardware, the technical challenges encountered by the multidisciplinary team shifted primarily toward software integration, creating realistic test environments, and managing web application state.

Challenge	Solution
Generating Realistic Sensor Behavior	Instead of dealing with physical sensor noise, the team had to recreate it. We developed a custom script that generated mock LDR values with injected mathematical "noise" to successfully test and verify the software's data smoothing and threshold logic.
Frontend Rendering Bottlenecks	The continuous influx of simulated real-time data initially caused the Next.js charts to lag. We resolved this by optimizing the API polling intervals and implementing data debouncing to balance real-time updates with browser performance limits.
Simulating Network Instability	To replicate the Wi-Fi drops that an ESP32 might experience in reality, we intentionally injected random connection failures into the simulation script. This allowed the team to implement and test robust error handling and offline UI fallback states in the web dashboard.

Table 5.1 Challenges and Solutions

6. Conclusions

This project successfully demonstrates the design and software implementation of an IoT-based smart lighting system equipped with real-time monitoring and control capabilities. Although physical hardware deployment was constrained due to component availability, the use of a comprehensive simulated environment validated the system's core automated, threshold-based logic. This logic is designed to effectively address the problem of unnecessary energy consumption. The seamless integration of the simulated edge-device data stream with modern web technologies (Next.js) resulted in a highly scalable, reliable, and efficient software architecture. Ultimately, this simulated prototype proves that the proposed smart lighting framework is robust. It confirms that the underlying software foundation is highly suitable for future physical deployment in smart homes and smart campus environments to enhance energy efficiency and user convenience.

7. Recommendations

Based on the project's simulated outcomes and current physical limitations, the following improvements are proposed for future work:

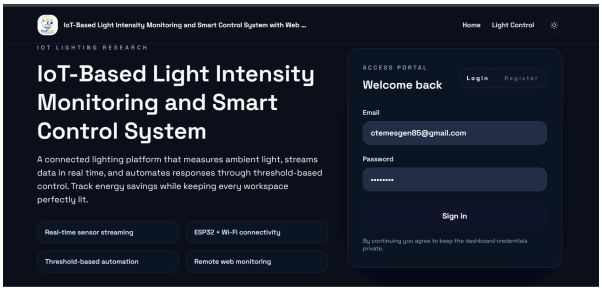
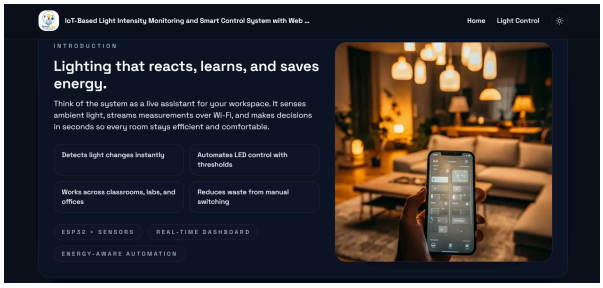
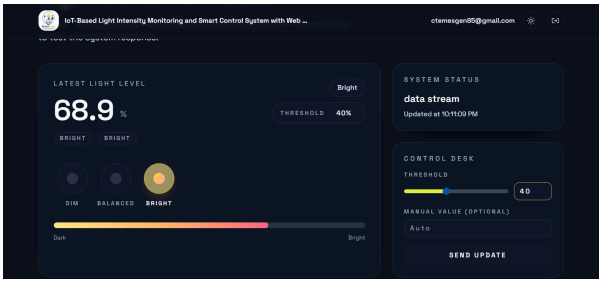
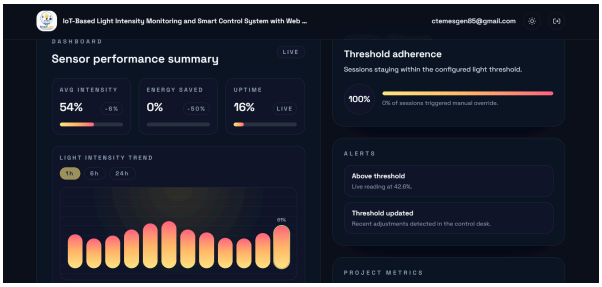
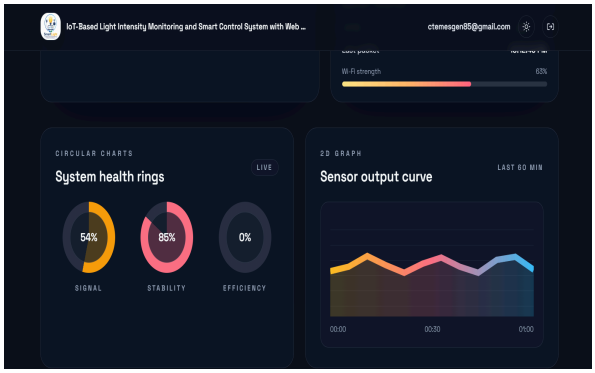
- **Hardware Integration:** The immediate next step is to procure the necessary physical components (ESP32 microcontrollers, LDR/BH1750 sensors, and LED actuators) to transition the system from a validated software simulation to a fully realized physical prototype.
- **Protocol Upgrade:** Implement MQTT alongside REST APIs to allow for even faster, lightweight real-time communication, which will be critical when scaling the physical hardware network.
- **Platform Expansion:** Add dedicated mobile application support (e.g., iOS/Android) to complement the web dashboard for improved user accessibility on the go.
- **Advanced Analytics:** Integrate machine learning algorithms to study user behavior and natural lighting patterns for predictive, rather than strictly threshold-based, control.
- **Scalability:** Once the hardware is physically deployed, expand the network to include multi-room monitoring to accommodate large-scale smart building or university campus applications.

8. References

- Brown, A. (2021). *Smart Home Technologies*.
- Espressif Systems. *ESP32 Documentation*.
- Smith, J. (2022). *IoT Systems and Applications*.
- Vercel. *Next.js Official Documentation*.

9. Appendices

Appendix A: API Endpoints

- GET /api/light Generates a simulated light intensity reading, derives the current status (Bright/Dark) against the active threshold, stores the record in memory, and returns the current state.
    - Response: value (number, percentage), threshold (number, percentage), status ("Bright" | "Dark"), and timestamp (ISO string).
- POST /api/light Handles system commands, such as threshold updates and manual overrides. Accepts an optional JSON payload to override the simulated reading and/or threshold. If provided, it updates the in-memory values, derives the new status, stores the record, and returns the updated state.
    - Request Body: { value?: number, threshold?: number }
    - Response: Same fields as GET /api/light.
- GET /api/light/history Returns an array of recent historical readings from storage, utilized to populate the interactive charts and data visualizations on the frontend dashboard.
    - Query Parameters: limit (optional integer; default is 50, minimum 1, maximum 200).
    - Response: { count: number, readings: LightSnapshot[] } (where each reading includes the value, threshold, status, and timestamp).

Appendix B: Screenshots